

DOG SQUAD: SQUIRREL SIEGE

動的難易度調整（DDA）× 機械学習 実装計画書

作成日 2026-06-15	対象リポジトリ github.com/imshota1009/dog-squad	Firebase Project dog-squad-game-app	現在フェーズ Phase 1 完了
----------------	---	-------------------------------------	----------------------

1. プロジェクト概要

DOG SQUAD のプレイデータを Firebase Firestore に蓄積し、機械学習モデルを訓練することで **動的難易度調整（Dynamic Difficulty Adjustment / DDA）** を実現する。プレイヤーの実力に合わせて敵のパラメータをリアルタイムで変化させ、「簡単すぎず、難しすぎない」最適な体験を自動で提供することを目指す。

2. データ収集計画

必要サンプル数の目安

サンプル数	モデル精度	判定
～50 セッション	ほぼルールベースと同じ	少なすぎ
100～200 セッション	簡単なパターンは学習できる	最低ライン
200～300 セッション	犬種・ステージ別の傾向も取れる	✔ 推奨
500+	かなり精度が上がる	理想

目標: 友達 5～10人 × 各 20～30 回プレイ（合計 100～300 セッション）

収集フィールド一覧（Firestore: dda_sessions コレクション）

フィールド	型	内容	ML での用途
stage	string	park / beach / snow	入力特徴量
upgrades	object	各アップグレードのレベル例: {"house":2,"missile":1}	入力特徴量（プレイヤー強化補正）
upgrade_total	int	全アップグレード合計レベル	入力特徴量（プレイヤー強化補正）
diff	string	easy / normal / hard / endless	入力特徴量
breed	string	選択した犬種	入力特徴量

win	bool	勝敗	ラベル / 特徴量
rank	string	S / A / B / C / -	目標変数（快適さ指標）
wave_reached	int	到達 WAVE 数	入力特徴量
wave_pct	int	全 WAVE 中の進行率 %	入力特徴量
score	int	スコア	入力特徴量
kills	int	撃破数	入力特徴量
max_combo	int	最大コンボ数	入力特徴量
clear_time	float	クリアタイム（秒）	入力特徴量
hp_remaining	int	犬小屋残 HP	快適さ指標
hp_pct	int	残 HP パーセンテージ	目標変数（メイン）
coins_earned	int	獲得コイン	参考
lang	string	ja / en	参考
ts	timestamp	サーバータイムスタンプ	時系列分析

💡 **快適さの定義:** hp_pct が 20～70% の範囲でクリア = 「ちょうど良い難しさ」とみなす。hp_pct > 70% = 簡単すぎ、hp_pct < 20% or 負け = 難しすぎ。この閾値を目標変数として使用する。

⚠️ **アップグレード補正:** ショップでアイテムを購入しているプレイヤーは upgrade_total が高くなる。モデルの学習時は upgrade_total を特徴量に含め、強化済みでも余裕があった場合と素の実力を区別する。

3. 実装フェーズ

PHASE 1

データ収集 ✓

完了

- game.js に saveDDASession() 追加
- Firestore dda_sessions コレクション作成
- セキュリティルール更新（write-only）
- Firebase にデプロイ済み

PHASE 2

モデル訓練

100～200 セッション後

- Firestore → CSV エクスポート
- Python (scikit-learn / TF) で訓練
- 入力: stage, diff, breed, score など
- 出力: 推奨難易度倍率 (0.6～1.8)
- TensorFlow.js 形式に変換

PHASE 3

ゲーム内推論

モデル完成後

- TF.js モデルをゲームに組み込み
- プレイ前に過去成績から倍率を推論
- 敵 HP / 速度 / 出現間隔に反映
- フォールバック: ルールベース維持
- A/B テストで効果を検証

4. 使用技術スタック

フェーズ

ツール / ライブラリ

用途

Phase 1	Firebase Firestore	セッションデータのリアルタイム蓄積
Phase 2	Python + pandas	データ前処理・特徴量エンジニアリング
Phase 2	scikit-learn / TensorFlow	回帰モデル訓練（難易度倍率予測）
Phase 2	tensorflowjs_converter	モデルを TF.js フォーマットに変換
Phase 3	TensorFlow.js	ブラウザ内リアルタイム推論
Phase 3	Firestore（読み取り）	プレイヤーの過去セッション参照

5. モデル設計（Phase 2 予定）

入力特徴量（X）

```
features = [  
    stage_encoded,      # park=0, beach=1, snow=2  
    diff_encoded,       # easy=0, normal=1, hard=2, endless=3  
    breed_encoded,      # shiba=0, golden=1, dal=2, corgi=3, husky=4, pug=5  
    avg_hp_pct,         # 過去 N セッションの平均残 HP%  
    avg_wave_pct,       # 平均進行率%  
    win_rate,           # 勝率  
    avg_score_norm,     # 正規化スコア  
]
```

出力（Y）

```
target = difficulty_multiplier # 0.6（大幅に易しく）～ 1.8（大幅に難しく）  
# hp_pct が 20～70% に収まるような倍率を学習
```

モデル候補

モデル	サンプル数	特徴
Random Forest Regressor	100～	解釈しやすい・過学習しにくい（最初の選択肢）
小規模ニューラルネット（3層）	200～	TF.js に直接変換できる
Gradient Boosting	300～	精度が高い・TF.js 変換に工夫が必要

6. Phase 2 に進むときの手順

1. Firebase コンソール → Firestore → **dda_sessions** コレクションを確認し、セッション数が 100 を超えていたら着手
2. Claude Code に「DDA の Phase 2 を始めてください」と声をかける
3. データエクスポートスクリプト・訓練スクリプト・変換スクリプトを順に実装
4. 生成された model.json + weights.bin を dog-squad/ml/ に配置
5. Phase 3: game.js に TF.js 推論コードを追加して Firebase にデプロイ

 Firestore でセッション数を確認する方法:

Firebase コンソール (console.firebase.google.com) → dog-squad-game-app → Firestore Database → dda_sessions コレクションを開く。右上にドキュメント数が表示される。

7. 関連ファイル一覧

ファイル	内容	状態
js/game.js	saveDDASession() 関数 — ゲーム終了時に Firestore 保存	✅ 実装済み
firestore.rules	dda_sessions: write-only ルール	✅ デプロイ済み
docs/DDA_plan.html / .pdf	本計画書 (アップグレード追記済み)	✅ 保存済み
docs/DDA_presentation.pptx	プレゼンテーション用スライド	✅ 保存済み
ml/export_data.py	Firestore → CSV エクスポートスクリプト	🟪 Phase 2
ml/train_model.py	モデル訓練・TF.js 変換スクリプト	🟪 Phase 2
ml/model.json + weights.bin	訓練済み TF.js モデル	🟪 Phase 2
js/dda.js	TF.js 推論 + 難易度適用ロジック	🟪 Phase 3